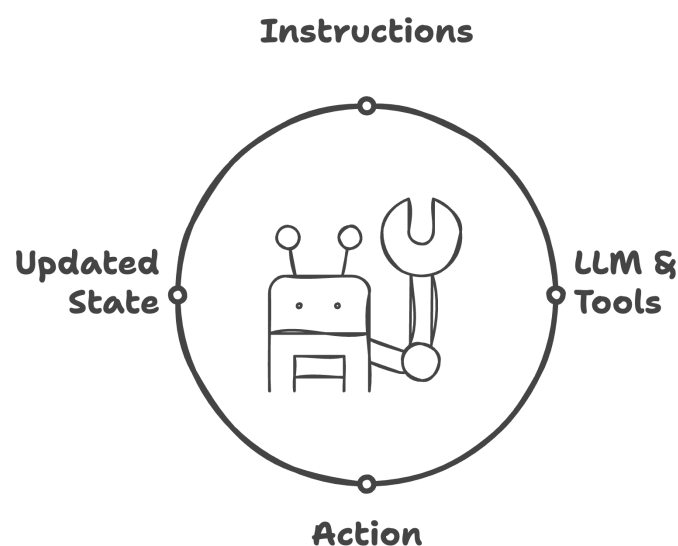


Creating an Agent

A core feature of agents is that they can use tools to find out more about the state of the world, or take action in response to instructions, without us needing to be explicit about how and when they do that.

Tools extend what the LLM can accomplish in the real world. As an example, let's craft a helper function so the agent can reach beyond pure text.



We'll create a tool for our agent to search the web for information. We'll use a service called Tavily, which is specifically designed to provide this kind of tool to agents. We can get a free API key from [Tavily](#).

! Note: Run the hidden cells below before running the rest of the code. !

3 hidden cells



Note: Make sure to set your `TAVILY_API_KEY` in the Environment Variables and connect the Environment Variables.

```
import os
tavily_api_key = os.environ["TAVILY_API_KEY"]
```

Tools in LlamaIndex are just regular Python functions, so they can do anything a regular function can.

When creating a tool, its very important to:

- give the tool a distinctive name, and a clear description using docstrings. The LLM uses the name and description to understand what the tool does.
- annotate the types. This helps the LLM understand the expected input and output types.
- use `async` when possible, since this will make the workflow more efficient.

How likely are you to recommend DataLab to a friend or co-worker?



Not at all likely

0

1

2

3

4

5

6

7

8

9

10

Extremely likely

powered by InMoment

```
# First, install the missing 'tavily' package
!pip install tavily

from tavily import AsyncTavilyClient

# Note the type annotations for the incoming query and the return string
async def search_web(query: str) -> str:
    """Useful for using the web to answer questions."""
    client = AsyncTavilyClient(api_key=tavily_api_key)
    return str(await client.search(query))
```

```
Requirement already satisfied: tavily in /home/repl/.venv-user/lib/python3.12/site-packages (1.1.0)
Requirement already satisfied: requests in /home/repl/.venv-user/lib/python3.12/site-packages (from tavily) (2.32.4)
Requirement already satisfied: aiohttp in /home/repl/.venv-user/lib/python3.12/site-packages (from tavily) (3.12.15)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (1.7.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (6.6.3)
Requirement already satisfied: propcache>=0.2.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (0.3.2)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiohttp->tavily) (1.20.1)
Requirement already satisfied: idna>=2.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from yarl<2.0,>=1.17.0->aiohttp->tavily) (3.10)
Requirement already satisfied: typing-extensions>=4.2 in /home/repl/.venv-user/lib/python3.12/site-packages (from aiosignal>=1.4.0->aiohttp->tavily) (4.16.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /home/repl/.venv-user/lib/python3.12/site-packages (from requests->tavily) (3.4.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/repl/.venv-user/lib/python3.12/site-packages (from requests->tavily) (2.5.0)
```

How likely are you to recommend DataLab to a friend or co-worker?

Not at all likely

0

1

2

3

4

5

6

7

8

9

10

Extremely likely

powered by InMoment



Instantiating an AgentWorkflow

An `AgentWorkflow` is our production line—it decides when to call each tool and how to route information between steps. After running the cell below we'll have the skeleton of an AI production line ready to roll.

With the tool and LLM defined, we can create an `AgentWorkflow` that uses the tool. `AgentWorkflow` has a special helper method for creating a single agent from a set of tools, so we'll use that.

We give it a system prompt that defines what the agent does. It's a good idea to tell the agent what kinds of things its tools will allow it to do.

! Note: Run the hidden cell below before running the rest of the code. !

1 hidden cell

```

from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.llms.openai import OpenAI
import os

llm = OpenAI(model="gpt-4o-mini", api_base=API_BASE)

workflow = AgentWorkflow.from_tools_or_functions(
    [search_web],
    llm=llm,
    system_prompt="You are a helpful assistant that answers questions. If you don't
know the answer, you can search the web for information.",
)

```



Running the Agent

Now that our agent is created, we can run it! Time to watch the agent bring your instructions to life. An `AgentWorkflow` expects to start with a question or prompt in the `user_msg`, which it passes to the agent.

How likely are you to recommend DataLab to a friend or co-worker?



Not at all likely

0

1

2

3

4

5

6

7

8

9

10

Extremely likely

powered by InMoment

```
response = await workflow.run(user_msg="What is the weather in San Francisco?")
print(str(response))
```

The current weather in San Francisco is as follows:

- **Temperature**: 18.5°C (65.3°F)
- **Condition**: Sunny
- **Wind**: 10.5 mph (16.9 kph) from the WSW
- **Humidity**: 70%
- **Pressure**: 1019 mb
- **Visibility**: 10 km
- **Chance of Rain**: 6%

Overall, it's a pleasant day with clear skies. For more details, you can check the [weather report](https://www.weatherapi.com/).

How likely are you to recommend DataLab to a friend or co-worker?

Not at all likely

0

1

2

3

4

5

6

7

8

9

10

Extremely likely

powered by InMoment